

Cahier des charges

Application de gestion de partitions pour un ensemble musical

I. Introduction

L'objectif de ce projet est de développer une plateforme multi-utilisateurs de gestion de partitions de pièce musicales.

A chaque partition pourra également être ajoutée un ou plusieurs type de médias (fichiers audio, fichiers PDF...).

La **maîtrise d'ouvrage** est le bureau d'une association gérant une chorale d'environ 60 personnes.

Le besoin est né de la difficulté à gérer les partitions pour une chorale (60 personnes regroupées en 4 pupitres avec des partitions différentes).

Cette plateforme se devra donc d'être collaborative, des utilisateurs spécifiques des ensembles musicaux pourront proposer des partitions.

Les fonctionnalités principales seront :

- création d'une "fiche morceau" et upload de partitions/fichiers audio pour un ou plusieurs instruments
- consultation de "fiches morceaux" permettant de retrouver tous les fichiers de partition
- possibilité de lire les fichiers multimédia si disponible directement à partir de la "fiche morceau"

I.1. Dates clefs

- Date de début de projet : **24/06/2026**
- Date de début de déploiement : **19/08/2026**
- Semaine de présentation : **semaine du 24/08/2026**

I.2. Modalités de travail

I.2.1. Méthodologie SCRUM

Vous pourrez base la gestion de votre projet sur une méthode Agile (par exemple Scrum) afin de répondre au mieux aux besoins clients et pouvoir rendre votre équipe flexible par rapport à d'éventuelles évolutions de demandes.

Les équipes sont constituées de 3 personnes et les sprints seront d'une durée de 2 semaines.

Toutes les 2 semaines une "sprint review" prenant la forme d'une présentation de 20 minutes maximum sera effectuée.

I.2.2. Outils de gestion de projet

Il vous est demandé de mettre en place plusieurs documents de suivi lors de ce projet :

- 1 planification globale sous la forme d'un **diagramme de Gantt**
- 1 outil de gestion de projet avec visualisation "**Kanban**" (par exemple, Github Project, Taiga ou Trello)

I.2.3. Livrables attendus

Les livrables attendus peuvent s'organiser la façon suivante :

- Gestion de projet :
 - vue Kanban de suivi de projet (faisant figurer les "**user stories**") ;
 - diagramme de Gantt de la planification du projet (par exemple avec GantProject).
- Cahier de spécifications fonctionnelles :
 - parcours utilisateurs (sous forme de **diagramme d'activité**) ;
 - diagrammes de **cas d'utilisation** ;
 - diagrammes de séquence et/ou d'activité **orienté métier**.
- Cahier de spécifications techniques :
 - diagrammes de **package** backend et/ou frontend (suivant la stack technique utilisée) ;
 - diagrammes de **classes** ;
 - diagrammes de séquence et/ou d'activité **orienté technique**.
- Plan de déploiement :
 - diagramme de **déploiement**.
- Intégration continue :

- **tests unitaires** sur le backend (test des endpoints) ;
- automatisation des test unitaires avec **pipeline Github Action**.
- Production :
 - **code source** des applications ;
 - **scripts de création de BDD** pour la production ;
 - fichiers de **configuration Docker**.
- Présentation du projet :
 - **diaporama de présentation du projet** (durée de présentation 45 minutes + 45 minutes de questions/réponses).

 Warning

Le diaporama de présentation du projet devra nécessairement comporter la présentation d'au moins 1 faille du OWasp Top Ten.

II. Expression des besoins

Les besoins exprimés ci-dessous devront pouvoir être transposés à plusieurs types de formation musicale :

- chorale
- orchestre
- groupe restreint
- duo
- ...

Ce document détaillera les besoins pour deux cas spécifiques mais il faut pouvoir se projeter sur les autres cas :

- gestion des partitions au sein d'une chorale ;
- gestion des partitions au sein d'un groupe de 4 personnes.

II.1. Cas de la chorale

L'expression du besoin détaillée par cette partie prend l'exemple d'un ensemble constitué de choristes (entre 40 et 60) et d'un/une chef.fe de chœur.

Le/la chef.fe de chœur est considéré.e comme responsable principal de la chorale.

Il/elle gère l'ensemble des chants, peut proposer un nouveau chant et propose un ensemble de documents de travail pour 1 chant (partition, audio, fichiers spécifiques...).

Les choristes sont répartis en 4 pupitres suivant leur tessitures de voix :

- soprano (voix aiguës féminines) ;
- alto (voix moyennes féminines) ;
- ténor (voix aiguës masculines) ;
- basse (voix graves masculines).

Le nombre de membres d'un pupitre est variable et peut évoluer au fil du temps.

Un chant chorale peut être composé de 4 voix (soprano, alto, ténor, basse). Chaque pupitre bénéficie d'un ensemble de documents de travail.

Les documents, pour la chorale, peuvent être sous différents formats :

- PDF ;
- image (PNG ou jpg) ;
- audio (mp3 ou ogg) ;
- Musescore.

Pour chacune de ces voix il peut y avoir un ou plusieurs fichiers pour 1 chant. Par exemples :

- 4 fichiers PDF séparés (1 pour chaque voix) ;
- 4 fichiers audio séparés (1 pour chaque voix) ;
- 1 fichier PDF regroupant toutes les voix.

Chaque pupitre peut consulter les documents qui lui sont propres mais également les documents des autres voix. Il n'est pas rare d'avoir un document (souvent PDF) qui fait figurer **toutes les voix**.

La plupart du temps, un fichier peut être proposé par le/la chef.fe de chœur mais il est possible qu'un membre de la chorale habilité puisse le faire. Nous pourrions appeler ce rôle spécifique "responsable de partition".

Seul un "responsable de partition" ou le/la chef.fe de chœur peuvent supprimer un document.

Jusqu'à présent les échanges se font par email et des documents sont régulièrement égarés.

Il est demandé de créer une plateforme permettant de faciliter le partage de partitions et ainsi facilement retrouver les documents.

II.2. Cas d'un groupe de 4 musicien.ne.s

Prenons un autre exemple, celui d'un groupe de 4 musicien.n.es (basse, guitare, batterie, voix).

Tout comme le cas de la chorale, 1 morceau peut avoir plusieurs documents.

Par exemple :

- 1 fichier audio pour la voix ;
- 1 partition PDF pour la batterie ;
- 1 fichier de tablature TuxGuitar pour basse et guitare.

Dans ce cas, tous les membres du groupe peuvent ajouter un document et tout le monde est "responsable de partition" et peut supprimer les fichiers.

Warning

Il est tout à faire possible qu'un utilisateur participe à plusieurs ensembles.

III. Spécifications fonctionnelles

Cette partie a pour objectif de détailler certains aspects fonctionnels de l'application.

i Info

Ces spécifications fonctionnelles ne sont pas définitives.

Vous pourrez les compléter en fonction de l'étude que vous allez mener.

III.1. Rôles des utilisateurs

Voici une liste des profils utilisateur de l'application :

- musicien.ne.s membre d'un ensemble souhaitant partager des partitions ;
- administrateur d'un enesmble (également considéré comme membre d'un ensemble) ;
- 1 super administrateur de toute l'application (détient tous les droits, ne peut être supprimé).

Tout utilisateur pourra accéder à l'application uniquement après authentification basée sur une connexion avec son adresse email et un mot de passe (cf. partie "Gestion des utilisateurs" pour la politique de gestion des mots de passe).

Pour chaque utilisateur, plusieurs informations devront être stockées, les informations obligatoires sont en **gras**.

Voici un liste des informations communes à tous les utilisateurs :

- **Nom** ;
- **Prénom** ;
- **E-mail** ;
- Photo de profil (facultative).

 Warning

Afin de respecter les recommandations de la RGPD seules les données nécessaires au fonctionnement de l'application devront être gardées.

Lors de l'inscription d'un utilisateur les conditions générales d'utilisation devront être indiquées clairement.

En vertu des règles du droit à l'oubli tout utilisateur pourra, s'il le souhaite et à tout moment, supprimer toutes les données qui le concernent.

III.2. Détails de certaines fonctionnalités

Info

Dans la suite de ce document les fonctionnalités accessibles avec un rôle “**administrateur**” seront indiquée par “**ADM**” et celles accessibles avec un rôle “**responsable de partition**” indiquées par **RESP**

Vous trouverez dans cette partie du cahier des charges une liste des fonctionnalités attendues pour le logiciel, ci-dessous un tableau récapitulatif.

Chaque fonctionnalité est accompagné de son indicateur MoSCoW:

Fonctionnalité	MoSCoW
Inscription d'un utilisateur	M
ADM Gestion d'ensemble	M
ADM / RESP Création d'une fiche morceau	M
Consultation d'une fiche morceau	M
ADM / RESP Ajout de document	M
Consultation de document	M
ADM / RESP Suppression de document	M
Ajout de commentaires sur un document	C
Ecouter les médias audio (mp3, ogg)	C
Ecouter/voir audio/vidéo en ligne	C
Ecouter les fichiers Muscores	C
Ecouter les fichiers TuxGuitar	W
Partager des morceaux entre ensembles	W

Dans la suite de ce document, vous trouverez des informations détaillées sur les différentes fonctionnalités “Must Have”.

III.2.1. Inscription

Toute personne pourra créer un compte à partir d'un formulaire de création de compte. L'utilisateur devra valider la création de son compte via le clic sur un lien unique de validation.

Par mesure de sécurité, tout mot de passe devra nécessairement suivre les règles suivantes :

- Longueur d'au moins 8 caractères ;
- Au moins 1 caractère spécial ;
- Au moins 1 majuscule.

Une fois le compte créé il pourra être invité par l'administrateur d'un ensemble.

III.2.2. Gestion d'un ensemble

Tout utilisateur pourra créer un ou plusieurs ensemble. Un ensemble pourra être constitué de plusieurs autres utilisateurs (pas de limite).

Ainsi, un ensemble peut tout aussi bien être un groupe constitué de 4 personnes qu'un orchestre de plusieurs dizaines de personnes.

Un ensemble sera caractérisé par un nom et une date de création ainsi qu'un ensemble d'instruments (voir partie "Gestion des morceaux et des partitions").

Le compte utilisé pour créer l'ensemble sera par défaut administrateur de cet ensemble.

Un fois un ensemble créée, l'utilisateur administrateur pourra inviter des utilisateurs en utilisant leur adresse email ou sélectionner un utilisateur pré-enregistré.

Une vue devra être accessible pour retrouver toutes les invitations envoyées et une indication de status devra être visible (actifs dans l'ensemble ou non).

Une invitation donnera lieu à l'envoi d'un email pour l'utilisateur ciblé.

Cet email contiendra un lien vers une vue permettant d'accepter l'inscription en **étant automatiquement associé à l'ensemble**.

i Info

Lors de la création d'une invitation par un administrateur d'ensemble, si l'utilisateur est déjà inscrit (et déjà associé à un ensemble) le lien contenu dans l'email envoyé ne redirigera pas vers le formulaire d'inscription mais permettra de valider l'inscription à un ensemble.

III.2.3. Création et consultation d'un morceau

Les utilisateurs définis par l'administrateur d'ensemble comme "responsable partition" peuvent ajouter un ou plusieurs morceaux. L'administrateur d'ensemble peut également ajouter des morceaux. La création d'un morceau est la première étape à respecter avant de pouvoir déposer un document.

Un morceau peut être caractérisé par les informations suivantes :

- titre de l'oeuvre ;
- compositeur/compositrice ;
- genre ;
- descriptif (optionnel).

Il devra être possible de déposer divers documents pour un morceau :

- fichier de partition (cf. partie ci-après) ;
- fichiers audio ajoutés par un utilisateur ;
- fichiers de partition de logiciel ;
- lien audio/vidéo intégré via Youtube.

Tout document devra être associé à **un ou plusieurs instruments**.

Les administrateur de l'ensemble et les responsable de partition pourront **modifier** et **supprimer** toute information.

Attention cependant, dans le cas d'une suppression ou modification d'information de morceau il faudra tracer et stocker en BDD l'opération.

III.2.4. Les différents documents

Pour chacun des médias ajoutés à une morceau, il sera important de stocker et d'afficher :

- la date d'ajout ;
- l'utilisateur à l'origine de l'ajout ;
- la ou les instruments concernés.

Seul l'utilisateur qui a ajouté le document et l'administrateur de l'ensemble devront pouvoir supprimer le document.

III.2.4.1. Fichiers de partition

Un fichier de partition peut être sous différents formats :

- PDF ;
- image (PNG ou jpg)
- Musescore ;
- TuxGuitar.

Fonctionnalité “**Could have**” et “**Won’t have this time but would like in the future**”: Dans le cas des partitions Musescore et TuxGuitar, il pourrait être intéressant d’intégrer un player permettant d’écouter le morceau (ceci permettrait de ne pas avoir à installer les logiciels associés).

III.2.4.2. Fichiers audio

Les fichiers audio pourront être sous la forme de mp3 ou d’ogg. Pour ces fichiers un lecteur pourra être intégré afin de pouvoir l’écouter sans avoir à le sauvegarder sur son disque.

III.2.4.3. Lien audio/vidéo

Tout comme les fichiers audio ajoutés par les utilisateurs, un lecteur intégré pourra être accessible pour un visionnage ou une écoute sans avoir à se rendre sur le site de l’hébergeur (par exemple Youtube).

IV. Spécifications techniques

⚠ Warning

Cette partie est une ébauche de cahier de spécifications techniques proposant une stack technique précise.

Il vous est tout à fait possible de proposer d'autres bibliothèques et/ou technologies. Les décisions doivent être validées par le "product owner".

IV.1. Stack technique

Ci-dessous un détail de la stack technique proposée :

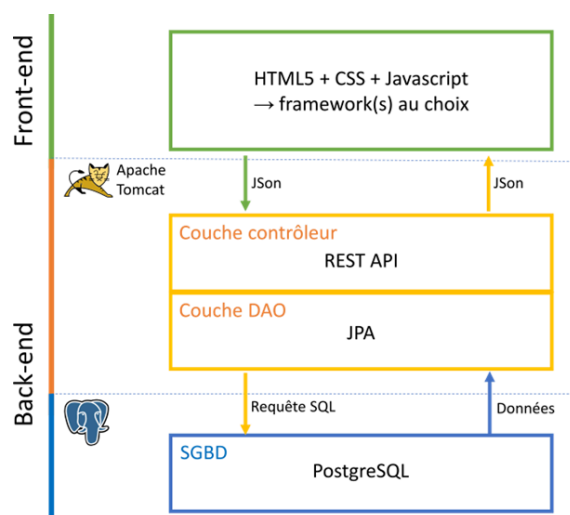


Figure 1: Image présentant la stack technique

Il vous est proposé d'utiliser le framework **Spring** pour le code côté serveur et la bibliothèque **React** pour le front-end.

IV.2. Bibliothèques de style CSS et de composants

Le choix des bibliothèques de développement de style CSS est libre mais doit suivre les contraintes suivantes :

- licence d'utilisation gratuite (attention cependant aux licences GPL qui imposent l'ouverture du code) ;
- bibliothèque stable pour garantir la pérennité du projet.

Vous pouvez, par exemple, choisir une bibliothèque de composants vous permettant de gagner en temps de développement, par exemple :

- ChakraUI ;
- Mantine ;
- shadcnUI.

Pour un aperçu des bibliothèques de composants existants vous pourrez consulter le site [React UI Library](#).

Warning

Intégrer une bibliothèque de composants pré-conçus peut présenter des difficultés. Il vous est tout à fait possible de faire l'intégralité des composants manuellement.

IV.3. Contraintes d'adaptabilité

L'interface de l'application développée devra s'adapter aux dimensions d'écran suivantes (largeurs en pixels) :

- Mobile :
 - de 380 à 600 px de large
- Tablette :
 - 601 à 1024 px de large
- Bureau :
 - 1025 à 1920 px de large

En version de bureau, l'interface devra être développée pour les navigateurs modernes suivants :

- Chrome v104 ou ultérieure ;
- Firefox v103 ou ultérieure ;
- Edge v103 ou ultérieure.

IV.4. Accessibilité

Il vous est demandé d'assurer l'accessibilité de l'application. Votre équipe veillera à concevoir une interface réfléchie en veillant à respecter les règles à cet égard (contraste, taille des composants, implémentation correcte du HTML...).

Pour vérifier l'accessibilité de votre site vous pourrez utiliser les outils suivants :

- [Google Lighthouse](#) (score de plus de 90 attendu) ;
- [Tanaguru](#).

IV.5. Sécurité de l'application

L'application se devra d'être robuste aux failles les plus communes reconnues et documentées par [OWASP](#) :

- Injection SQL ;
- Injection Javascript ;

- Cross-site request forgery;
- Hachage des mots de passe en bdd ;
- restriction d'accès à la base de données (utilisation de vues et d'utilisateurs aux accès restreints)

⚠ Warning

Le diaporama de présentation du projet devra nécessairement comporter la présentation d'au moins 1 faille du OWasp Top Ten.

IV.6. Code de la solution

Le code des logiciels devra être hébergé sur des dépôts Git. Il vous est fortement conseillé de séparer les projets en plusieurs dépôts Git

- 1 dépôt pour l'interface graphique ;
- 1 dépôt pour le/les webservices développés ;
- 1 dépôt global accueillant les 2 autres.

Vous pourrez, par exemple, imbriquer tous vos dépôt en tant que submodules.

Le code de la solution devra être documenté en utilisant les outils de documentation propre aux technologies utilisées, par exemple :

- Javadoc pour le code Java ;
- JSDoc pour le code Javascript/Typescript.